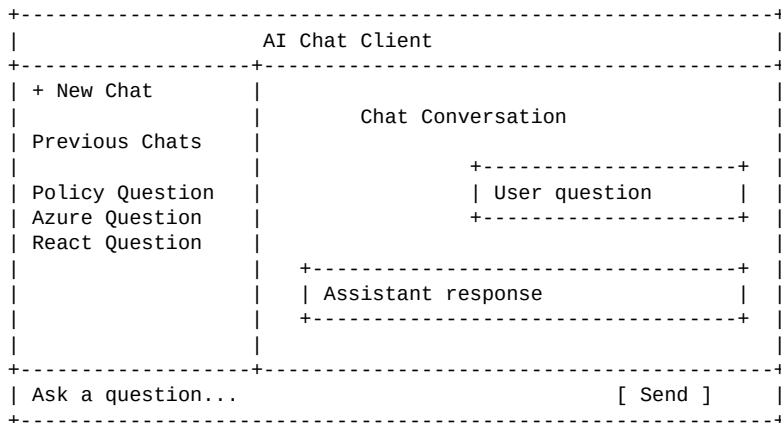


Turning the assistant-ui React Client into a Reusable Chat Template

Because the current assistant-ui project is working well, it makes sense to evolve it into a reusable React TypeScript chat-client template before adding Corporate Policy Assistant-specific functionality.

The proposed capabilities - New Chat, previous conversations, and distinct user/assistant message styling - are generic chat features and belong in the reusable foundation.

Recommended Target Layout



Version 1 Capabilities

- 1. New Chat button** - Starts a new conversation/thread while keeping older conversations available.
- 2. Previous Chat list** - Organize history as previous conversations rather than individual requests. A conversation can contain many user questions and assistant responses. Clicking a conversation should restore the entire thread.
- 3. User and Assistant message styling** - Visually distinguish roles. A common pattern is right-aligned user messages and left-aligned assistant responses. Assistant responses can use larger cards because they may contain headings, lists, citations, tables, and code.

Additional Features to Design For

- Automatically generated conversation titles based on the first user question.
- Delete and rename conversation.
- Copy response.
- Regenerate/retry response.
- Stop generating when streaming is introduced.
- Markdown rendering for assistant responses.
- Code blocks with copy buttons.

- Loading/thinking indicator.
- Error and retry UI.
- Responsive/collapsible conversation sidebar.
- Future support for attachments, feedback, speech/dictation, and suggestions.

Keep the Base Template Generic

Do not put authentication, Azure-specific code, RAG, Azure AI Search, Microsoft Foundry, corporate branding, or policy-specific controls into the base template yet. Those capabilities should be layered on top of the generic chat client when building the Corporate Policy Assistant.

Recommended Architecture

```

Reusable Assistant-UI Chat Template
|
+-- App Shell
|
+-- Sidebar
|   +-- New Chat
|   +-- Conversation History
|
+-- Thread
|   +-- User Message
|   +-- Assistant Message
|   +-- Composer
|
+-- Runtime
|   +-- Model Adapter
|   +-- Thread List
|   +-- History Adapter
|
+-- Backend Adapter
    +-- Fake AI                <- current implementation
    +-- Azure Function         <- Corporate Policy Assistant
    +-- Foundry Agent
    +-- OpenAI API
    +-- Other AI backend
  
```

Why the Backend Adapter Boundary Matters

The current fake modelAdapter can become an intentional interface boundary. The reusable UI should not need to know whether an answer comes from Microsoft Foundry, an Azure Function, AWS Bedrock, OpenAI, a RAG service, or a temporary test response. This separation makes the template useful for many future AI applications.

Recommended Next Milestone: Chat Template v0.2

New Chat -> Conversation Sidebar -> Select Previous Chat -> User/Assistant Message Styling -> Preserve Conversations

Once these capabilities work cleanly, save or copy the project as the baseline assistant-ui React TypeScript chat template. The Corporate Policy Assistant can then be built from that template instead of using the policy application itself to experiment with basic chat behavior.

After Chat Template v0.2, conversation persistence is the logical next implementation step.